

Behavior Driven Development

Projet Jeux

Durée : 7h

Type : Travail individuel noté

Contexte

Votre équipe de développement vient de se voir commander le développement d'une API en C# (dll) permettant de gérer des parties pour plusieurs jeux. L'idée étant que cette librairie encapsule la gestion d'une partie et la prise en compte des règles de chaque jeu : gestion des joueurs, des tours, évolution des scores, fin de partie, etc...

Nous aimerions avoir les jeux suivants dans cette librairie :

- TicTacToe (morpion)
- Partie de tennis (deux sets gagnants)
- Fléchettes
- Mastermind
- [Bonus] Bowling (deux joueurs)
 - <https://bcc85.fr/calcul-du-score/>

Travail demandé

En vous basant sur les règles de chacun des jeux choisis (**3 minimum**) et en les analysant, vous allez devoir :

- 1) Identifier et créer les scénarios Gherkin correspondants, puis implémenter les tests et le code de production correspondant en respectant le cycle BDD.
- 2) Rédiger une documentation et justification de vos choix techniques (voir détails dans la section livrables).

Je vous conseille fortement, et en rapport avec le cycle BDD, de passer plus de temps à la réflexion de vos scénarios et stratégie qu'à l'implémentation (cf la documentation demandée).

Important :

- Comme pour les TP précédents, pour simplifier la mise en œuvre nous travaillerons avec des projets de type libraires .Net Standard sans affichage.
- Comme pour les TP précédents, j'accorde le plus d'importance à la qualité de vos scénarios et vos choix.
- **[Bonus]** Selon votre avancement, et votre aisance technique, vous pouvez ajouter en plus un projet console, graphique WPF ou un front end à votre solution, qui utilisera votre librairie en conditions « réelles » : récupération des données utilisateurs, appels à votre librairie et affichage des résultats.

Livrables

- 1) Un lien vers un repo Git contenant :
 - a) Vos projets de tests et votre projet contenant le code de l'API demandée.
 - b) Pour plus de lisibilité, vous pouvez organiser vos scénarios en utilisant plusieurs fichiers .feature pour chacun des jeux. Côté code de la librairie, une classe minimum pour chaque jeu.
- 2) Documentation (**readme.md**) et justification des choix techniques. En complément de votre implémentation, vous devrez produire un document de justification (3-5 pages) structuré comme suit :
 - a) Analyse et justification des scénarios
 - i) **Identification des cas de test** : Argumentez votre stratégie de couverture des cas de test pour chaque jeu (cas nominaux, cas limites, cas d'erreur).
 - ii) **Priorisation des scénarios** : Justifiez l'ordre de développement de vos scénarios et expliquez quels sont les scénarios critiques vs. secondaires.
 - b) Architecture et représentation des données
 - i) **Lisibilité des données de test** : Argumentez vos choix pour la présentation des données dans les fichiers .feature (utilisation de tables, exemples, background).
 - ii) **Extensibilité** : Expliquez comment votre architecture facilite l'ajout de nouveaux jeux ou la modification des règles existantes.
 - c) Stratégie BDD et bonnes pratiques
 - i) **Langage ubiquitaire** : Démontrez comment vos scénarios utilisent le vocabulaire métier de chaque jeu.
 - ii) **Réutilisabilité** : Justifiez vos choix de step definitions communs vs. spécifiques.
 - iii) **Maintenance** : Expliquez comment votre organisation de code facilite la maintenance et l'évolution.